Efficient Hardware Architecture Design for Rotary Position Embedding of Large Language Models

Wenjie Li, Gang Wang, Dongxu Lyu, Ningyi Xu, and Guanghui He, *Member, IEEE*

***Abstract*—Due to the substantial demands of storage and computation imposed by large language models (LLMs), there has been a surge of research interest in their hardware acceleration. As a technique involving non-linear operations, rotary position embedding (RoPE) has been adopted by some recently released LLMs. However, there is currently no reported research on its hardware design. This paper, for the first time, presents an efficient hardware architecture design for RoPE of LLMs. We first explore the similarities between RoPE and the coordinate rotation digital computer (CORDIC) algorithm, while also considering the commonly used quantization scheme for LLMs. Additionally, we propose a hardware-friendly solution to address the issue of excessively large input angle ranges. Then we present a CORDIC-based approximation for RoPE and develop a hardware architecture for it. The experimental results demonstrate that our design can save up to 45.7% area cost and 31.0% power consumption when compared with the fixed-point counterpart, while maintaining almost the same model performance. Compared to the straightforward implementation using floating-point arithmetic, our design can reduce up to 91.4% area cost and 88.9% power consumption, with negligible performance loss.**

***Index Terms*—Large language models, rotary position embedding, hardware architecture, CORDIC, quantization.**

I. Introduction

THANKS to the success of ChatGPT [1], large language models (LLMs) have caused a tremendous sensation and become ubiquitous in our daily lives. The unparalleled performance of LLMs has garnered widespread attention from both academic and industrial communities, positioning them as one of the most prominent topics in artificial intelligence (AI) research today [2]. Numerous LLM families such as GPT3 [3], GPT4 [4], OPT [5], PaLM [6], PaLM2 [7], LLaMA [8], LLaMA2 [9], and Falcon [10] have been introduced in the past few years. Almost all of them are built upon Transformer architecture [2], [11], [12]. As the name indicates, the model size of an LLM is usually over billions of parameters, leading to intensive computational and memory requirements. In this context, hardware acceleration has aroused the interest of researchers. Numerous works related to LLM accelerators have been reported in the literature [13], [14], [15], [16], [17], [18]. For LLMs, inference is invoked more frequently than training. Therefore, this paper focuses on LLM inference.

In natural language understanding, sequential order of words is of great importance. Therefore, position embedding is employed to inject positional information into LLMs. Compared to other positional embedding techniques such as absolute position embedding [11], relative position embedding [19], and ALiBi [20], rotary position embedding (RoPE) [21] has been more widely adopted in recently released LLMs such as PaLM, PaLM2, LLaMA, LLaMA2, and Falcon. In RoPE, rotation transformation is applied to query and key vectors in the attention modules of LLMs. Each pair of elements in the query and key vectors is considered as a new vector. It will be rotated by an angle which is a product of its position index and a constant. In other words, the query vector and the key vector at the same position will both be multiplied by a matrix whose elements are the cosine and sine of the rotation angles. As a result, the relative position information is encoded into the inner product between the query vector and the key vector.

Coordinate rotation digital computer (CORDIC) algorithm is a hardware-efficient iterative method which is typically used to calculate trigonometric functions such as cosine and sine [22]. Some researchers have devised its variant, referred to as hyperbolic CORDIC [23], [24], which can be used for the calculation of logarithms, exponentials, and square roots. The key idea of CORDIC algorithm is to iteratively rotate a vector to approach a desired angle. Because of its efficient hardware architecture, the CORDIC algorithm has been widely applied in various hardware designs. In this paper, we consider the conventional CORDIC algorithm [22]. Instead of using CORDIC algorithm to compute the cosine and sine required by RoPE, we proposed a CORDIC-based approximation for all the computations involved in RoPE.

When it comes to LLMs, whether in software or hardware implementations, quantization is always an inevitable topic [18], [25], [26], [27], [28]. It is a technique that maps a large set of values into a smaller set, aiming to reduce bitwidths and therefore storage requirements. On the other hand, lower bitwidths can also simplify the hardware complexity of arithmetic units. Almost all the reported works on LLM accelerators [13], [14], [15], [16], [17], [18] have employed quantization schemes that quantize the weights and activations into fixed-point numbers. By means of a co-design

Received 26 November 2024; revised 13 February 2025; accepted 24 March 2025. Date of publication 31 March 2025; date of current version 16 June 2025. This work was supported in part by the Smart Grid-National Science and Technology Major Project under Grant 2024ZD0802100 and in part by the National Natural Science Foundation of China under Grant 92464302. This article was recommended by Guest Editor J. Park. *(Corresponding authors: Ningyi Xu; Guanghui He.)*

Wenjie Li and Ningyi Xu are with the School of Electronic Information and Electrical Engineering, Shanghai Jiao Tong University, Shanghai 200240, China (e-mail: wenjielinju@sina.com; xuningyi@sjtu.edu.cn).

Gang Wang, Dongxu Lyu, and Guanghui He are with the State Key Laboratory of Micro/Nano Engineering Science, School of Electronic Information and Electrical Engineering, Shanghai Jiao Tong University, Shanghai 200240, China (e-mail: wangganginJSTU@sjtu.edu.cn; lvdongxu@sjtu.edu.cn; guanghui.he@sjtu.edu.cn).

Digital Object Identifier 10.1109/JETCAS.2025.3556443

with quantization, the hardware complexity of our design is further reduced.

In this paper, we present an efficient hardware architecture design for RoPE of LLMs. Inspired by the rotational nature of RoPE, we explore its similarities with the CORDIC algorithm. Besides, the commonly used quantization scheme is taken into account. The input angles of RoPE have a wide range, whereas the CORDIC algorithm only supports a limited input angle range. A hardware-friendly solution is proposed to address this issue. Subsequently, we propose a CORDIC-based approximation for RoPE, which initializes the inputs of the CORDIC algorithm using the elements of either the query or key vectors. The scaling factor of the CORDIC algorithm can be fused into that of the quantization scheme, successfully eliminating the need for multiplication. Furthermore, a hardware architecture is developed for our proposed CORDIC-based approximation. Demonstrated by the experimental results, our design can outperform the straightforward implementation in terms of area cost and power consumption, while maintaining almost the same model performance.

The main innovations of this paper can be summarized as follows.

- The similarities between the RoPE and CORDIC algorithms are explored in this paper, which have not been reported in the current literature.
- Based on the computational characteristics of the RoPE input angles, we propose a hardware-friendly solution to address the issue of excessively large input angle ranges.
- We propose to fuse the scaling factors of the CORDIC algorithm and the quantization scheme, successfully eliminating the need for multiplication.
- We develop a hardware architecture for RoPE that offers lower area cost and power consumption than the straightforward implementations. Our work is the first in the literature on the hardware design of RoPE.

The rest of the paper is organized as follows. Section II presents the background and preliminaries. In Section III, the hardware architecture design for RoPE is proposed. Then experimental results and analysis are given in Section IV to demonstrate the effectiveness and superiority of our work. Finally, Section V draws the conclusion.

II. Background and Preliminaries

In this section, we first provide an overview for RoPE of LLMs. Then we briefly review the CORDIC algorithm and the fundamentals of quantization.

A. Rotary Position Embedding (RoPE)

Multi-head attention (MHA) [2], [11], [12] has been adopted in the attention modules of all the reported Transformer-based LLMs. For the sake of brevity and without loss of generality, the discussions throughout this paper only consider the query and key vectors within a single head. Note that our design works for all the heads of MHA and its variants: multi-query attention (MQA) [29] and grouped-query attention (GQA) [30].

Query vectors $\mathbf{q}_i$ are multiplied by key vectors $\mathbf{k}_j$ to obtain inner products, where i and j are their position indices, subject to the constraint $i \geq j$. Denote the context length (maximum supported number of tokens) of an LLM by N, and then $i, j = 0, 1, 2, \cdots, N-1$. In addition, we denote the number of dimensions of $\mathbf{q}_i$ and $\mathbf{k}_j$ as $2D$. Note that our denotation differs from the convention by an additional factor of 2. For example, LLaMA2-7b and Falcon-7b have $D = 64$ and $D = 32$, respectively. Again, we emphasize that the discussions throughout this paper only consider the query and key vectors within a single head. Here, the dimension number is specific to one head.

RoPE is proposed in [21], integrating positional information into the query and key vectors. It regards every two adjacent elements of the query and key vectors as a new vector and rotates it by an angle. The angle is a product of the position index and a constant. Formally, the rotation transformation for a vector $\mathbf{x}$ is shown in (1), at the bottom of the page. In RoPE, such transformation is applied to both query and key vectors by setting $\mathbf{x} = \mathbf{q}_i$ and $\mathbf{x} = \mathbf{k}_i$, respectively. Obviously, each angle is a product of the position index i and a constant θ_j, for $i = 0, 1, \cdots, N-1$ and $j = 0, 1, \cdots, D-1$. Given a specific LLM, we have $\theta_j = b^{-j/D}$, where b is a constant base which is typically set to 10000. For example, LLaMA2-7b has $\theta_j = 10000^{-j/64}$, for $j = 0, 1, \cdots, 63$. After applying rotation transformation to $\mathbf{q}_i$ and $\mathbf{k}_j$, their inner product will contain the relative position information. Namely, there exists a function g such that

$$(\mathbf{q}_i^{(R)})^T \mathbf{k}_j^{(R)} = T(\mathbf{q}_i)^T T(\mathbf{k}_j) = g(\mathbf{q}_i, \mathbf{k}_j, i-j). \tag{2}$$

$\mathbf{q}_i^{(R)} = T(\mathbf{q}_i)$ and $\mathbf{k}_i^{(R)} = T(\mathbf{k}_j)$ denote the transformed query and key vectors, respectively. For more details about RoPE, please refer to [21]. In this paper, we focus on its hardware implementation rather than its mathematical properties.

B. Cordic Algorithm

In CORDIC algorithm, a two-dimension vector is rotated several times to approach a desired angle. After a rotation of

$$T(\mathbf{x}) = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_{2D-2} \\ x_{2D-1} \end{bmatrix} \odot \begin{bmatrix} \cos i\theta_0 \\ \cos i\theta_0 \\ \cos i\theta_1 \\ \cos i\theta_1 \\ \vdots \\ \cos i\theta_{D-1} \\ \cos i\theta_{D-1} \end{bmatrix} + \begin{bmatrix} -x_1 \\ x_0 \\ -x_3 \\ x_2 \\ \vdots \\ -x_{2D-1} \\ x_{2D-2} \end{bmatrix} \odot \begin{bmatrix} \sin i\theta_0 \\ \sin i\theta_0 \\ \sin i\theta_1 \\ \sin i\theta_1 \\ \vdots \\ \sin i\theta_{D-1} \\ \sin i\theta_{D-1} \end{bmatrix} = \begin{bmatrix} x_0 \cos i\theta_0 - x_1 \sin i\theta_0 \\ x_1 \cos i\theta_0 + x_0 \sin i\theta_0 \\ x_2 \cos i\theta_1 - x_3 \sin i\theta_1 \\ x_3 \cos i\theta_1 + x_2 \sin i\theta_1 \\ \vdots \\ x_{2D-2} \cos i\theta_{D-1} - x_{2D-1} \sin i\theta_{D-1} \\ x_{2D-1} \cos i\theta_{D-1} + x_{2D-2} \sin i\theta_{D-1} \end{bmatrix} \tag{1}$$

Algorithm 1 Original CORDIC Algorithm

Input: Vector $\begin{bmatrix} x_0 \\ y_0 \end{bmatrix}$, desired angle γ $(-\pi/2 \le \gamma \le \pi/2)$, and iteration number I;

Output: Vector $\begin{bmatrix} x \\ y \end{bmatrix}$ where $x \approx (x_0 \cos\gamma - y_0 \sin\gamma)/s$ and $y \approx (x_0 \sin\gamma + y_0 \cos\gamma)/s$;

$\Delta\gamma_0 \Leftarrow \gamma$;
$\delta_0 \Leftarrow (-1)^{\text{Sign}(\Delta\gamma_0)}$;
for $i = 0, 1, \cdots, I-1$ **do**
  $x_{i+1} \Leftarrow x_i - \delta_i 2^{-i} y_i$;
  $y_{i+1} \Leftarrow y_i + \delta_i 2^{-i} x_i$;
  $\Delta\gamma_{i+1} \Leftarrow \Delta\gamma_i - \delta_i \tan^{-1}(2^{-i})$;
  $\delta_{i+1} \Leftarrow (-1)^{\text{Sign}(\Delta\gamma_{i+1})}$;
$\begin{bmatrix} x \\ y \end{bmatrix} \Leftarrow \begin{bmatrix} x_I \\ y_I \end{bmatrix}$;

an angle γ, a vector $\begin{bmatrix} x \\ y \end{bmatrix}$ will become

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos\gamma & -\sin\gamma \\ \sin\gamma & \cos\gamma \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} x\cos\gamma - y\sin\gamma \\ x\sin\gamma + y\cos\gamma \end{bmatrix}. \quad (3)$$

Denote the number of iterations by I. The desired angle γ is approximated by rotating the input vector I times, for which the rotation angles are $\gamma_0, \gamma_1, \cdots, \gamma_{I-1}$. In other words, we have $\gamma \approx \gamma_0 + \gamma_1 + \cdots + \gamma_{I-1}$ and

$$\begin{bmatrix} x' \\ y' \end{bmatrix} \approx \prod_{i=0,1,\cdots,I-1} \begin{bmatrix} \cos\gamma_i & -\sin\gamma_i \\ \sin\gamma_i & \cos\gamma_i \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}. \quad (4)$$

In practical applications, the absolute values of the angles are set to some constants, denoted as $c_0, c_1, \cdots, c_{I-1}$. However, their sign bits may differ. The underlying idea is that we can rotate the vector back, once the accumulated angle has exceeded the desired value. Accordingly, the rotation angle at the ith iteration is

$$\gamma_i = \delta_i c_i = (-1)^{\text{Sign}\left(\gamma - \sum_{j=0,1,\cdots,i-1}\gamma_j\right)} c_i, \quad (5)$$

where $\delta_i = (-1)^{\text{Sign}\left(\gamma - \sum_{j=0,1,\cdots,i-1}\gamma_j\right)}$ is the sign bit indicator and $\text{Sign}(x)$ is the sign bit of x: 0 for positive and 1 for negative.

Furthermore, (4) can be rewritten as

$$\begin{bmatrix} x' \\ y' \end{bmatrix} \approx \prod_{i=0,1,\cdots,I-1} \cos\gamma_i \begin{bmatrix} 1 & -\tan\gamma_i \\ \tan\gamma_i & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}. \quad (6)$$

For the sake of efficient hardware implementation, c_i is typically set to $\tan^{-1}(2^{-i})$ such that $\tan(\gamma_i) = \tan(\delta_i c_i) = \delta_i 2^{-i}$ and $\cos\gamma_i = \cos c_i = \cos(\tan^{-1}(2^{-i}))$. As a result, we have

$$\begin{bmatrix} x' \\ y' \end{bmatrix} \approx s \prod_{i=0,1,\cdots,I-1} \begin{bmatrix} 1 & -\delta_i 2^{-i} \\ \delta_i 2^{-i} & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}, \quad (7)$$

where $s = \prod_{i=0,1,\cdots,I-1} \cos c_i$ is a constant which we refer to as the scaling factor of CORDIC algorithm. Alg. 1 presents the CORDIC algorithm which computes $x \approx (x_0 \cos\gamma - y_0 \sin\gamma)/s$ and $y \approx (x_0 \sin\gamma + y_0 \cos\gamma)/s$. To obtain $\cos\gamma$ and $\sin\gamma$, one can initialize Alg. 1 with $\begin{bmatrix} x_0 \\ y_0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ and multiply the outputs by the scaling factor s. Note that with large I, we have $c_0 + c_1 + \cdots + c_{I-1} \approx 1.74$. Therefore we restrict the input angle to the first quadrant, i.e., $-\pi/2 \le \gamma \le \pi/2$.

C. Fundamentals of Quantization

In this paper, we consider uniform quantization that maps floating-point numbers into integers (fixed-point numbers), since it is the mainstream quantization scheme that has been widely adopted in LLMs [18], [25], [26], [27], [28]. Given a floating-point scaling factor S and an integer zero-point Z, a floating-point number a_f is quantized into a b-bit integer a_q as follows:

$$a_q = \text{clamp}\left(\text{round}\left(\frac{a_f}{S}\right) + Z, a_{min}, a_{max}\right). \quad (8)$$

a_{max} and a_{min} are the maximum and minimum values that can be represented by a b-bit integer. If signed integers are allowed, we have $a_{max} = 2^{b-1} - 1$ and $a_{min} = -2^{b-1}$. Otherwise, $a_{max} = 2^b - 1$ and $a_{min} = 0$. After RoPE, the transformed query and key vectors, i.e., $\mathbf{q}_i^{(R)}$ and $\mathbf{k}_i^{(R)}$, will be quantized using (8).

The majority of the reported works on LLM accelerators [13], [14], [15], [16], [17], [18] have employed quantization schemes that quantize the weights and activations into integers. One of the primary reasons is that the area cost and power consumption of fixed-point arithmetic units are lower than those of their floating-point counterparts.

III. Proposed Hardware Architecture Design

In this section, we first present the proposed CORDIC-based approximation for RoPE. Then, we develop its hardware architecture and provide some details on the dataflow.

A. Cordic-Based Approximation

As RoPE regards two adjacent elements of $\mathbf{q}_i$ and $\mathbf{k}_i$ as a vector and rotates it by an angle, it exhibits inherent similarity with the CORDIC algorithm. As shown in (1), every two adjacent elements can be obtained by performing Alg. 1 and multiplying the outputs x and y by the scaling factor s. For example, perform Alg. 1 by initializing $\begin{bmatrix} x_0 \\ y_0 \end{bmatrix}$ with $\begin{bmatrix} q_{i,2j} \\ q_{i,2j+1} \end{bmatrix}$ and γ with $i\theta_j$, for $j = 0, 1, \cdots, D-1$. The outputs will be as follows:

$$x = \frac{q_{i,2j}\cos i\theta_j - q_{i,2j+1}\sin i\theta_j}{s},$$
$$y = \frac{q_{i,2j+1}\cos i\theta_j + q_{i,2j}\sin i\theta_j}{s}. \quad (9)$$

Next we consider quantization and RoPE together. Taking $\mathbf{q}_i$ as an example for discussion, the same applies to $\mathbf{k}_i$. As analyzed above, we can obtain $\mathbf{q}_i^{(R)}/s$ when performing Alg. 1 D times with appropriate initialization. Then, it will be multiplied by s and then be quantized via (8). By fusing s and S together, we construct a new scaling factor $S' = S/s$. The multiplications corresponding to s can be eliminated by replacing S with S'. We refer to this technique as scaling factor fusion.

In the existing quantization schemes for LLMs, fusing scaling factors is a common technique. For example, [26] proposes to fuse the scaling factors of the activation matrices into previous normalization or linear layer. Our scaling factor

fusion is inspired by these works. However, these works only fuse the scaling factors in the network quantization, whereas our scaling factor fusion simultaneously considers the CORDIC-approximated RoPE and network quantization. Although our design employs the CORDIC algorithm, it is not mandatory. For instance, sine and cosine can also be directly computed using Synopsys DesignWare IP cores. The choice of the CORDIC algorithm leads to the presence of scaling factor s. In conclusion, scaling factor fusion is a unique innovation in our design and should not be regarded as a simple application of previous techniques.

Note that we assume the input angles are restricted to $[-\pi/2, \pi/2]$ in the above discussions. However, the input angles may be particularly large in RoPE. For example, LLaMA2 supports a context length of 4K, implying that the maximum value of the input angle is $4095\theta_0 = 4095$. Unfortunately, the angles that the CORDIC algorithm can support are much smaller than 4095. As we have mentioned before, its maximum supported angle is roughly 1.74. In other words, we have to address the issue of exceedingly large input angles if the CORDIC algorithm is employed for RoPE.

In the straightforward implementation of RoPE, the cosine and sine are calculated directly and then multiplied by the elements of the query and key vectors. In addition to the CORDIC algorithm, we can also utilize the cosine and sine IP cores from the DesignWare library of Synopsys. However, according to their datasheet, the input angles of the fixed-point cosine and sine IP cores are also restricted to the range of $[0, 2]$ (unsigned) or $[-1, 1]$ (signed). In other words, this issue is also intractable for the straightforward implementation with fixed-point arithmetic.

For a large input angle γ, we decompose it as $\gamma = \gamma' + \frac{\pi}{2}l$, where $-\pi/2 < \gamma' < \pi/2$ and l is a non-negative integer. Rotating a vector by an angle of γ is equivalent to rotating it first by an angle of γ' and then by an angle of $\frac{\pi}{2}l$. The rotation process can be formulated as (10), shown at the bottom ofthe page. With the properties of trigonometric functions, we further have

$$\begin{bmatrix} x'' \\ y'' \end{bmatrix} = \Psi(x', y', l) = \begin{cases} \begin{bmatrix} x' & y' \end{bmatrix}^T, & \text{if } l\%4 = 0; \\ \begin{bmatrix} -y' & x' \end{bmatrix}^T, & \text{if } l\%4 = 1; \\ \begin{bmatrix} -x' & -y' \end{bmatrix}^T, & \text{if } l\%4 = 2; \\ \begin{bmatrix} y' & -x' \end{bmatrix}^T, & \text{else,} \end{cases} \tag{11}$$

where $\begin{bmatrix} x' \\ y' \end{bmatrix}$ is the vector obtained by rotating $\begin{bmatrix} x \\ y \end{bmatrix}$ by an angle of γ'. A straightforward implementation for decomposing γ into $\gamma' + \frac{\pi}{2}l$ is not hardware-efficient. A divider is required to divide γ by $\frac{\pi}{2}$ to obtain the integer quotient l and the remainder γ'. Next we propose a hardware-friendly solution to circumvent the undesired division and also the multiplication between the position index and the constant θ_j.

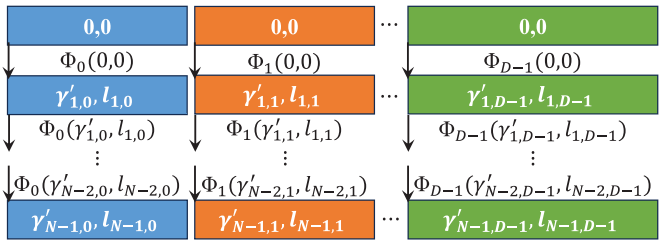


Fig. 1. The computation process of the input angles in a serial manner.

Denote the input angle by $\gamma_{i,j}$, which is the product of its corresponding position index i and a constant θ_j: $\gamma_{i,j} = i\theta_j$, as shown in (1). For the first token ($i = 0$), all the angles are zero. For the second token, we have $\gamma_{1,j} = \theta_j = b^{-j/D} \leq 1$ (note that b is always a large positive integer like 10000), for $j = 0, 1, \cdots, D-1$. Obviously, the input angles for the second token inherently satisfy the constraint of being within the range $(-\pi/2, \pi/2)$. Construct a function $\Phi_j(\gamma, l)$ for $-\pi/2 < \gamma < \pi/2$ and $l = 0, 1, 2, 3$, such that

$$\Phi_j(\gamma, l) = \begin{cases} \left[\gamma + \theta_j \;\; l \right], & \text{if } \gamma + \theta_j < \pi/2; \\ \left[\gamma + \theta_j - \pi/2 \;\; (l+1)\%4 \right], & \text{else.} \end{cases} \tag{12}$$

Consequently, the input angles for the third token can be obtained by applying Φ_j on those for the second one. Therefore, for $j = 0, 1, \cdots, D-1$, we have

$$\left[\gamma'_{i,j} \;\; l_{i,j} \right] = \begin{cases} \left[0 \;\; 0 \right] & \text{if } i = 0; \\ \Phi_j(\gamma'_{i-1,j}, l_{i-1,j}), & \text{else.} \end{cases} \tag{13}$$

Fig. 1 illustrates the computation process of the input angles in a serial manner. Note that the iterative updating rule for the input angles is designed after exploring the mathematical relationship between input angles at adjacent positions. Such mathematical relationship is specific to RoPE. In other applications of CORDIC algorithm, the input angles are likely to be random, rather than following regular patterns similar to those in (1), i.e., $\gamma_{i,j} = i\theta_j = ib^{-j/D}$.

Alg. 2 presents our proposed CORDIC-based approximation for RoPE. The scaling factors are fused off-line which will not introduce additional computational overhead during inference. In contrast, it eliminates the multiplications corresponding to s. It can be seen that neither multiplication nor division is required by our proposed CORDIC-based approximation, if scaling factor fusion is applied. The input angle $\gamma'_{i,j}$ computed in Step 2.1 is restricted to $(0, \pi/2)$. Thus it can be directly used as the input of the CORDIC algorithm. In this algorithm, we assume that the inputs involve N tokens, for convenience. However, in some cases, only a portion of query and key vectors need to be processed. For example, in the generation stage of LLMs, only one query vector is considered at each

$$\begin{bmatrix} x'' \\ y'' \end{bmatrix} = \begin{bmatrix} \cos\gamma & -\sin\gamma \\ \sin\gamma & \cos\gamma \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} \cos\frac{\pi}{2}l & -\sin\frac{\pi}{2}l \\ \sin\frac{\pi}{2}l & \cos\frac{\pi}{2}l \end{bmatrix} \begin{bmatrix} \cos\gamma' & -\sin\gamma' \\ \sin\gamma' & \cos\gamma' \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} \cos\frac{\pi}{2}l & -\sin\frac{\pi}{2}l \\ \sin\frac{\pi}{2}l & \cos\frac{\pi}{2}l \end{bmatrix} \begin{bmatrix} x' \\ y' \end{bmatrix}. \tag{10}$$

Algorithm 2 CORDIC-Based Approximation for RoPE

Input: Query vectors $\mathbf{q}_i$ and key vectors $\mathbf{k}_i$ $(i = 0, 1, \cdots, N-1)$, constants θ_j $(j = 0, 1, \cdots, D-1)$, number of iteration I, and indicator for scaling factor fusion F;

Output: Transformed query vectors $\mathbf{q}_i^{(R)} = T(\mathbf{q}_i)$ and transformed key vectors $\mathbf{k}_i^{(R)} = T(\mathbf{k}_i)$ $(i = 0, 1, \cdots, N-1)$;

// **Step 1**: Compute the scaling factor s and fuse it into S if $F = 1$ (it can be done off-line rather than during inference)

$s \Leftarrow \prod_{i=0,1,\cdots,I-1} \cos(\tan^{-1}(2^{-i}))$;

if $F = 1$ **then**

  Fuse s with the quantization scaling factor S of $\mathbf{q}_i$ and $\mathbf{k}_i$, for $i = 0, 1, \cdots, N-1$;

// **Step 2**: Performed during inference

for $i = 0, 1, \cdots, N-1$ **do**

  for $j = 0, 1, \cdots, D-1$ **do**

    // **Step 2.1**: Compute the input angle

    if $i = 0$ **then**

      $\begin{bmatrix} \gamma'_{i,j} & l_{i,j} \end{bmatrix} \Leftarrow \begin{bmatrix} 0 & 0 \end{bmatrix}$;

    else

      $\begin{bmatrix} \gamma'_{i,j} & l_{i,j} \end{bmatrix} \Leftarrow \Phi_j(\gamma'_{i-1,j}, l_{i-1,j})$; // see (12)

    // **Step 2.2**: Perform the rotation transformation

    $\begin{bmatrix} x_q \\ y_q \end{bmatrix} \Leftarrow \text{CORDIC}(\begin{bmatrix} q_{i,2j} \\ q_{i,2j+1} \end{bmatrix}, \gamma'_{i,j}, I)$;

    $\begin{bmatrix} x_k \\ y_k \end{bmatrix} \Leftarrow \text{CORDIC}(\begin{bmatrix} k_{i,2j} \\ k_{i,2j+1} \end{bmatrix}, \gamma'_{i,j}, I)$;

    // Multiplying s if scaling factor fusion is not applied

    if $F = 0$ **then**

      $\begin{bmatrix} x_q \\ y_q \end{bmatrix} \Leftarrow \begin{bmatrix} x_q s \\ y_q s \end{bmatrix}$;

      $\begin{bmatrix} x_k \\ y_k \end{bmatrix} \Leftarrow \begin{bmatrix} x_k s \\ y_k s \end{bmatrix}$;

    // **Step 2.3**: Post processing

    $\begin{bmatrix} q_{i,2j}^{(R)} \\ q_{i,2j+1}^{(R)} \end{bmatrix} \Leftarrow \Psi(x_q, y_q, l_{i,j})$; // see (11)

    $\begin{bmatrix} k_{i,2j}^{(R)} \\ k_{i,2j+1}^{(R)} \end{bmatrix} \Leftarrow \Psi(x_k, y_k, l_{i,j})$; // see (11)

time step. At each times step, the number of newly generated query vectors is equal to that of newly generated key vectors (in the generation stage, previous transformed key vectors have been stored). All the cases can be supported by changing the value of N in Alg. 2. Besides, it is noteworthy that scaling factor fusion is optional.

B. Hardware Architecture and Dataflow

For the CORDIC algorithm, we employ an unrolled architecture as depicted in Fig. 2. It is referred to as unrolled CORDIC unit (UCU) which is comprised of I parts corresponding to the I iterations. The multipliers corresponding to s can be removed if scaling factor fusion is applied. A UCU can compute two adjacent elements of $\mathbf{q}_i^{(R)}$ or $\mathbf{k}_i^{(R)}$ within one cycle. As aforementioned, one can use the original CORDIC algorithm to compute cosine and sine. Hence, a UCU can output $\cos\gamma$ and $\sin\gamma$ by feeding it with $\begin{bmatrix} x_0 \\ y_0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$.

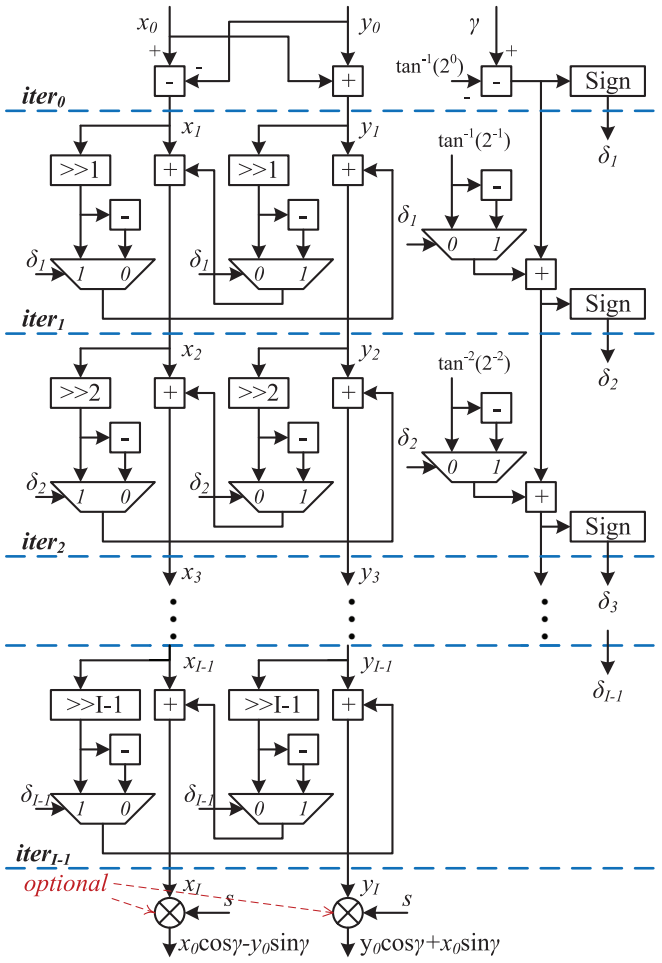


Fig. 2. Architecture of the unrolled CORDIC unit (UCU).

As for the straightforward implementation, Fig. 3 depicts the straightforward architecture to compute the adjacent elements of $\mathbf{q}_i^{(R)}$ or $\mathbf{k}_i^{(R)}$. For convenience, we refer to it as straightforward unit (SFU). For SFU, we assume that the input angle $\gamma = i\theta_j$ is available. The Cos&Sin unit is deployed to simultaneously generate the cosine and sine of γ. With fixed-point arithmetic, a UCU can work as the Cos&Sin unit when the input is initialized as $\begin{bmatrix} x_0 \\ y_0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$. We denote a UCU fed with $\begin{bmatrix} x_0 \\ y_0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ by $\text{UCU}_{\text{fixed}}$. Similar to Alg. 2, the scaling factors can be fused. In this case, the corresponding multipliers can be removed from $\text{UCU}_{\text{fixed}}$. It is also feasible to implement the Cos&Sin unit by directly employing a cosine IP core and a sine IP core from the DesignWare library of Synopsys.

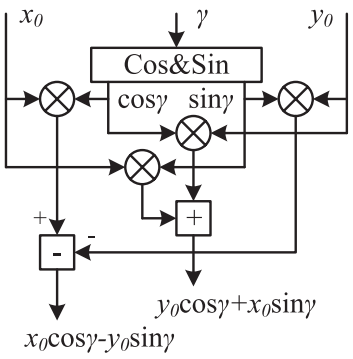


Fig. 3. Architecture of SFU, which is a straightforward architecture to compute the adjacent elements of $\mathbf{q}_i^{(R)}$ or $\mathbf{k}_i^{(R)}$.

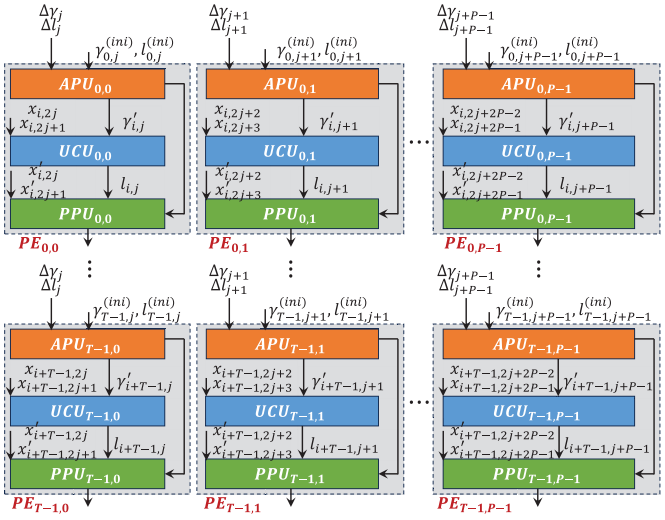
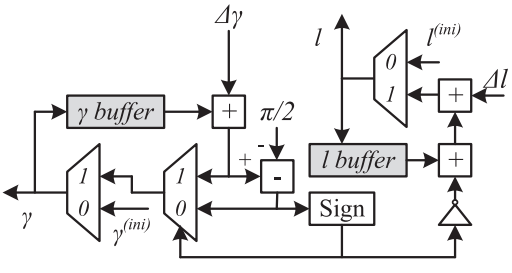
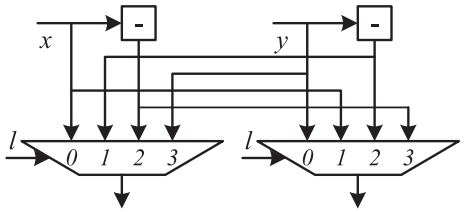


Fig. 4. Overall hardware architecture for our proposed CORDIC-based approximation algorithm.

In practice, a hardware accelerator is devised to simultaneously handle multiple tokens and multiple dimensions. Suppose T tokens and $2P$ dimensions are handled simultaneously. The overall hardware architecture for our proposed CORDIC-based approximation algorithm is given in Fig. 4. It is composed of $T \times P$ processing units (PEs), each of which contains an angle processing unit (APU), a UCU and a post-processing unit (PPU). The architectures of APU and PPU are depicted in Fig. 5. APU is responsible for processing the input angles, while PPU is employed to perform (11). They correspond to Step 2.1 and Step 2.3 of Alg. 2, respectively. The PEs in the same row correspond to the same token, i.e., the same position index. Each column corresponds to two adjacent dimensions. Generally, we have $T \leq N$ and $P \leq D$. A fully parallel architecture will be obtained if we set T and P to N and D, respectively. Unlike the SFU of Fig. 3 which contains four multipliers, our proposed architecture contains no multiplier if scaling factor fusion is applied. Even though scaling factor fusion is not applied, the multipliers corresponding to s has lower complexity than that of SFU, since s is a constant. With fixed-point arithmetic, SFU fails to address the issue of exceedingly large input angle ranges.



(a) APU.



(b) PPU.

Fig. 5. Architectures of angle processing unit (APU) and post-procesing unit (PPU).

It is necessary to introduce our proposed APU and PPU into SFU.

In last subsection, we have discussed the computation process of the input angles in a serial manner. To simultaneously handle multiple tokens and dimensions, the computation of input angles needs to be adjusted to a parallel manner. Consider initial γ and l:

$$\begin{aligned} \gamma'_{i,j} &= \gamma_{i,j}^{(ini)} = i\theta_j - \lfloor 2i\theta_j/\pi \rfloor \times \pi/2, \\ l_{i,j} &= l_{i,j}^{(ini)} = \lfloor 2i\theta_j/\pi \rfloor \% 4, \end{aligned} \tag{14}$$

for $i = 0, 1, \cdots, T-1$ and $j = 0, 1, \cdots, D-1$. They are constants for the first T tokens. For the next T tokens, we have $\gamma_{i,j} = \gamma_{i-T,j} + T\theta_j$, for $i = T, T+1, \cdots, 2T-1$ and $j = 0, 1, \cdots, D-1$. Let

$$\Delta\gamma_j = T\theta_j - \lfloor 2T\theta_j/\pi \rfloor \times \pi/2, \ \Delta l_j = \lfloor 2T\theta_j/\pi \rfloor \% 4, \tag{15}$$

for $j = 0, 1, \cdots, D-1$. Further construct a function $\Phi_j^{(T)}(\gamma, l)$ which is a T-parallel version of $\Phi_j(\gamma, l)$:

$$\begin{aligned} &\Phi_j^{(T)}(\gamma, l) \\ &= \begin{cases} \left[\gamma + \Delta\gamma_j \ (l + \Delta l_j)\%4 \right], & \text{if } \gamma + \Delta\gamma_j < \pi/2; \\ \left[\gamma + \Delta\gamma_j - \pi/2 \ (l + \Delta l_j + 1)\%4 \right], & \text{else.} \end{cases} \end{aligned} \tag{16}$$

For $j = 0, 1, \cdots, D-1$, we have

$$\begin{aligned} &\left[\gamma'_{i,j} \ l_{i,j} \right] \\ &= \begin{cases} \left[\gamma_{i,j}^{(ini)} \ l_{i,j}^{(ini)} \right], & \text{if } i = 0, 1, \cdots, T-1; \\ \Phi_j^{(T)}(\gamma'_{i-T,j}, l_{i-T,j}), & \text{else,} \end{cases} \end{aligned} \tag{17}$$

which is a T-parallel version of (13). In the parallel manner, we first initialize $\left[\gamma'_{i,j} \ l_{i,j} \right]$ using $\left[\gamma_{i,j}^{(ini)} \ l_{i,j}^{(ini)} \right]$ for the first T tokens, i.e., for $i = 0, 1, \cdots, T-1$. Then (17) is used to iteratively compute the following $\left[\gamma'_{i,j} \ l_{i,j} \right]$.

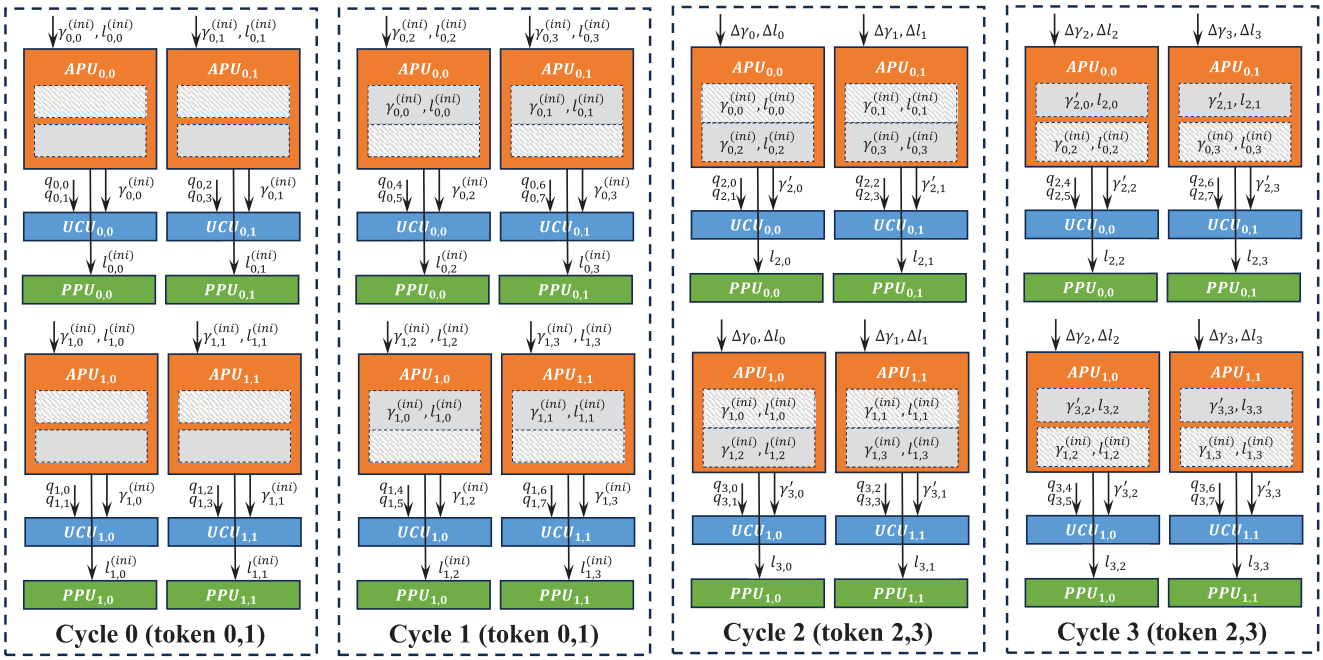


Fig. 6. The input arrangement (in part) and storage status of APU for a toy example where $T = P = 2$, $N = 4$, $D = 4$ and rotation transformation is applied on $\mathbf{q}_{i,j}$.

Now we present the dataflow in the T-parallel manner. At the beginning, $\text{APU}_{i,j}$ is fed with $\gamma_{i,j}^{(ini)}$ and $k_{i,j}^{(ini)}$, for $i = 0, 1, \cdots, T-1$ and $j = 0, 1, \cdots, P-1$. At the same time, $\gamma_{i,j}^{(ini)}$ is output and stored in the γ buffer, while $l_{i,j}^{(ini)}$ is output and stored in the l buffer. In the next cycle, the computation proceeds to the next $2P$ dimensions. $\text{APU}_{i,j}$ is fed with $\gamma_{i,j+P}^{(ini)}$ and $l_{i,j+P}^{(ini)}$, for $i = 0, 1, \cdots, T-1$ and $j = 0, 1, \cdots, P-1$. Also, $\gamma_{i,j+P}^{(ini)}$ is output and stored in the γ buffer, while $l_{i,j+P}^{(ini)}$ is output and stored in the l buffer. After $\lceil D/P \rceil$ cycles, all the $2D$ dimensions have been processed, and the computation proceeds to the next T tokens, i.e., tokens with position indices $T, T+1, \cdots, 2T-1$. Then, for $i = 0, 1, \cdots, T-1$ and $j = 0, 1, \cdots, P-1$, $\gamma'_{i,j} = \gamma_{i,j}^{(ini)}$ and $l_{i,j} = l_{i,j}^{(ini)}$ are fetched from the buffers. $\text{APU}_{i,j}$ performs $\Phi_j^{(T)}$ on them and outputs $\gamma'_{i+T,j}$ and $l_{i+T,j}$, which will be also stored in the γ and l buffers, respectively. In the next cycle, $\gamma'_{i,j+P} = \gamma_{i,j+P}^{(ini)}$ and $l_{i,j+P} = l_{i,j+P}^{(ini)}$ are fetched from the buffers, for $i = 0, 1, \cdots, T-1$ and $j = 0, 1, \cdots, P-1$. $\text{APU}_{i,j}$ performs $\Phi_{j+P}^{(T)}$ on them and outputs $\gamma'_{i+T,j+P}$ and $l_{i+T,j+P}$, which will be also stored in the γ and l buffers, respectively. This procedure repeats until all the tokens have been processed.

Fig. 6 visualizes the input arrangement (in part) and storage status of APUs for a toy example where $T = P = 2$, $N = 4$, $D = 4$ and rotation transformation is applied on $\mathbf{q}_{i,j}$. In this example, each APU is capable of storing two $\gamma'_{i,j}$ and two $l_{i,j}$, since $D/P = 2$. After the first two cycles, $\left[\gamma'_{i,j}\ l_{i,j}\right]$ has been initialized by $\left[\gamma_{i,j}^{(ini)}\ l_{i,j}^{(ini)}\right]$ and stored in the buffers, for $i = 0, 1$ and $j = 0, 1, 2, 3$. In the next two cycles, they are fetched from the buffers to compute $\left[\gamma'_{i,j}\ l_{i,j}\right]$ for $i = 2, 3$ and $j = 0, 1, 2, 3$. For example, $\text{APU}_{0,0}$ fetches $\left[\gamma'_{0,0}\ l_{0,0}\right] = \left[\gamma_{0,0}^{(ini)}\ l_{0,0}^{(ini)}\right]$ and computes $\left[\gamma'_{2,0}\ l_{2,0}\right] = \Phi_0^{(2)}(\gamma'_{0,0}, l_{0,0})$ in cycle 2.

IV. Evaluation and Analysis

A. Experimental Settings

We consider three recently released open-source LLMs that adopt RoPE as their position embedding technique: LLaMA2-7b [9], LLaMA2-13b [9], and Falcon-7b [10]. Model performance is first evaluated on two language modeling tasks: WikiText2 (WIKI) [31] and C4 [32]. Moreover, five zero-shot tasks are also considered: Arc-e [37], ARC-c [37], BoolQ [38], Winogrande [39], and OpenBookQA [40]. These tasks have been considered in many studies related to LLMs [18], [26], [33], [34], [35], [36]. We use the same evaluation methodology as that provided in [26]. All experiments were based on the source code provided in [26] and were conducted using the PyTorch [45] framework with HuggingFace [46] integration. When we evaluate RoPE on a particular model (using approximation or fixed-point quantization), the other parts of the model remain unchanged, still using the floating-point operations as in the original source code.

In hardware evaluation, we consider three RoPE designs: the proposed one (RoPE_{pro}), the fixed-point counterpart ($\text{RoPE}_{\text{fixed}}$) and the straightforward one with floating-point arithmetic ($\text{RoPE}_{\text{float}}$). For a fair comparison, all the three designs use the same overall architecture given in Fig. 4. They differ only in the PE architecture. Fig. 7 depicts illustrative block diagrams for the PEs of the three RoPE architectures and the relationships between different units. To distinguish them from each other, we add subscripts to their names. As a fixed-point counterpart of our proposed RoPE_{pro}, $\text{RoPE}_{\text{fixed}}$ also incorporates our proposed APU_{pro} to address the issue of excessively large input angle ranges. The $\text{APU}_{\text{float}}$ of $\text{RoPE}_{\text{float}}$ includes only a 32-bit floating-point multiplier for computing $i \times \theta_j$, as the floating-point DesignWare IP cores for computing

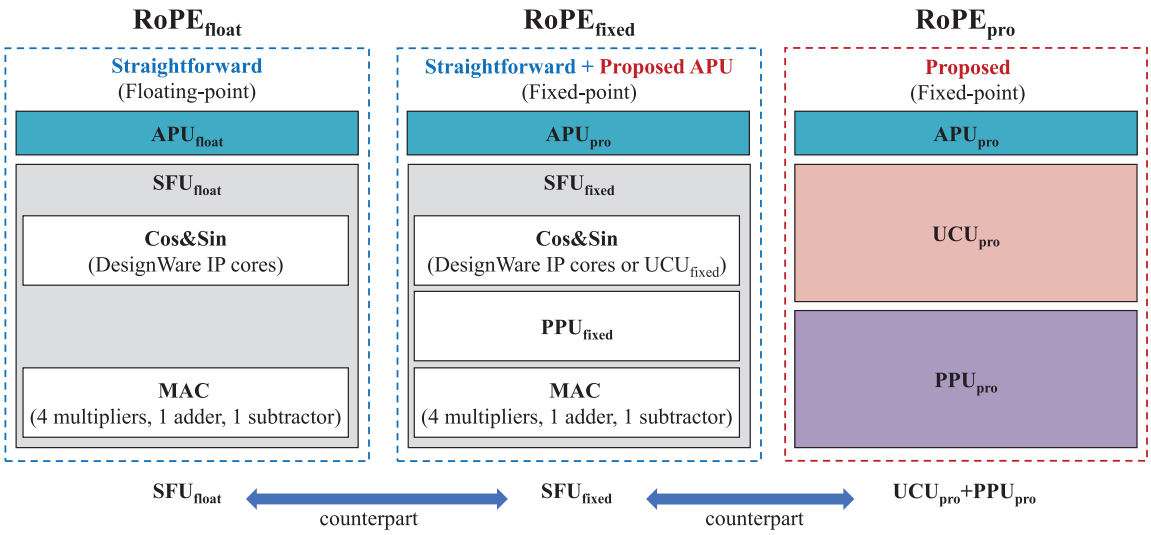


Fig. 7. Illustrative block diagrams for the PEs of different RoPE architectures and the relationships between different units.

TABLE I
BITWIDTH CONFIGURATIONS (SIGN,INT,FRAC) FOR ROPE$_{PRO}$ AND SFU$_{FIXED}$

Our proposed architecture					SFU$_{fixed}$ with UCU$_{fixed}$	
x_i, y_i in UCU	γ in UCU	γ in APU	$\Delta\gamma$	$\tan^{-1}(2^{-i})$	x_i, y_i	UCU$_{fixed}$
(1,5,8)	(1,1,6)	(0,1,15)	(0,1,15)	(0,0,7)	(1,5,8)	See Table II

TABLE II
BITWIDTH CONFIGURATIONS (SIGN,INT,FRAC) FOR UCU$_{FIXED}$

x_1, y_1	x_2, y_2	x_3, y_3	x_4, y_4	Other x_i, y_i	γ	$\tan^{-1}(2^{-i})$
(0,1,0)	(1,1,1)	(1,1,3)	(1,1,5)	(1,1,9)	(1,1,6)	(0,0,7)

cosine and sine impose no restrictions on the input angles. Both SFU$_{float}$ and SFU$_{fixed}$ use the architecture shown in Fig. 3, with the only difference being that SFU$_{fixed}$ inserts a PPU$_{fixed}$ into the output of the Cos&Sin unit. For the Cos&Sin unit of SFU$_{fixed}$, UCU$_{fixed}$ serves as an alternative to the fixed-point DesignWare IP cores. Excluding the APU, both SFU$_{float}$ and SFU$_{fixed}$ are counterparts of UCU$_{pro}$ plus PPU$_{pro}$ (denoted as UCU$_{pro}$ + PPU$_{pro}$).

Since our design is orthogonal to specific quantization schemes, we adopt naive quantization scheme for RoPE$_{pro}$. Namely, we quantize the floating-point inputs of RoPE$_{pro}$ into the nearest fixed-point numbers. The same quantization scheme is applied to RoPE$_{fixed}$. Table I lists the bitwidth configurations used in RoPE$_{pro}$ and SFU$_{fixed}$. Table II presents the bitwidth configurations for UCU$_{fixed}$. The bitwidths are presented in the form of (sign,int,frac). Since $\begin{bmatrix} x_0 \\ y_0 \end{bmatrix}$ is initialized by $\begin{bmatrix} 1 \\ 0 \end{bmatrix}$ in UCU$_{fixed}$, x_i and y_i in the first several iterations can be quantized using lower bitwidths. APU$_{pro}$ employed in RoPE$_{fixed}$ is the same as that of RoPE$_{pro}$. If the Cos&Sin unit of SFU$_{fixed}$ is implemented using the fixed-point DesignWare IP cores, the bitwidths of its input and outputs are the same as those of UCU$_{fixed}$. Note that the main goal of this paper is not to investigate the minimum bitwidths. The bitwidth configurations of Table I and Table II are adopted for minimizing performance loss. Regarding RoPE$_{float}$, its APU$_{float}$ is composed of a 32-bit floating-point multiplier. Its Cos&Sin unit is implemented using 32-bit floating-point DesignWare IP cores. The MAC unit of RoPE$_{float}$ is implemented using 16-bit floating-point arithmetic. The bitwidths used in RoPE$_{float}$ follow those used in the software implementation from the Hugging Face Transformers repository.[1]

We implement the hardware architectures in Verilog HDL and synthesize them using Synopsys Design Compiler with a TSMC 65nm CMOS technology library, in order to estimate area and power. The voltage is set to 1.08 V. To demonstrate the acceleration effect, we also compare the hardware performance of our design with that of GPU. The GPU model used in our experiments is the NVIDIA GeForce RTX3090Ti. We perform rotation transformation on the query and key vectors of a large number of tokens, and calculate the average time taken for rotation transformation on a single vector.

B. Model Performance

The perplexity scores (lower is better) are present in Table III, Table IV and Table V. As SFU$_{fixed}$ with UCU$_{fixed}$ is not the focus of our work, we only consider LLaMA2-7b in its evaluation for the sake of simplicity. All the three LLMs have been involved in the experiments for our proposed design. It shows that 8 iterations are sufficient for no performance loss, while the performance loss brought by 5 ~ 7 iterations

[1]Please refer to the source code modeling_llama.py and modeling_falcon.py in Hugging Face Transformers repository.

TABLE III

PERPLEXITY (↓) ACHIEVED BY THE ORIGINAL MODELS ($\text{RoPE}_{\text{FLOAT}}$)

Task \ Model	LLaMA2-7b	LLaMA2-13b	Falcon-7b
WIKI	5.47	4.88	6.59
C4	6.97	6.47	9.50

TABLE IV

PERPLEXITY (↓) ACHIEVED BY RoPE_{PRO} AND $\text{RoPE}_{\text{FIXED}}$ WITH AND WITHOUT QUANTIZATION IN WIKI

# of iterations	4	5	6	7	8
RoPE_{pro}					
LLaMA2-7b	5.55	5.50	5.48	5.47	5.47
LLaMA2-7b (quan.)	5.55	5.50	5.48	5.48	5.48
LLaMA2-13b	4.95	4.91	4.89	4.89	4.88
LLaMA2-13b (quan.)	4.96	4.92	4.90	4.89	4.89
Falcon-7b	6.65	6.61	6.59	6.59	6.59
Falcon-7b (quan.)	6.65	6.61	6.60	6.59	6.59
$\text{SFU}_{\text{fixed}}$ with $\text{UCU}_{\text{fixed}}$					
LLaMA2-7b	5.55	5.50	5.48	5.47	5.47
LLaMA2-7b (quan.)	5.56	5.50	5.48	5.48	5.48

TABLE V

PERPLEXITY (↓) ACHIEVED BY RoPE_{PRO} AND $\text{RoPE}_{\text{FIXED}}$ WITH AND WITHOUT QUANTIZATION IN C4

# of iterations	4	5	6	7	8
RoPE_{pro}					
LLaMA2-7b	7.05	7.00	6.98	6.98	6.97
LLaMA2-7b (quan.)	7.05	7.00	6.98	6.98	6.97
LLaMA2-13b	6.52	6.49	6.48	6.47	6.47
LLaMA2-13b (quan.)	6.52	6.49	6.48	6.47	6.47
Falcon-7b	9.58	9.53	9.51	9.50	9.50
Falcon-7b (quan.)	9.57	9.53	9.51	9.51	9.51
$\text{SFU}_{\text{fixed}}$ with $\text{UCU}_{\text{fixed}}$					
LLaMA2-7b	7.05	7.00	6.98	6.98	6.97
LLaMA2-7b (quan.)	7.05	7.00	6.98	6.98	6.97

is negligible. Moderate performance loss is introduced by 4 iterations. However, it is still smaller than 0.1. On the other hand, the performance loss brought by our adopted quantization scheme for RoPE is also negligible.

The model performance is also evaluated in the zero-shot tasks. The model accuracy is given in Table VI. For the sake of simplicity, only three numbers of iterations are considered: 4, 6, and 8. In contrast to the language modeling tasks, the zero-shot tasks exhibit better robustness to our CORDIC-based RoPE approximation. While slight accuracy loss is observed in some cases, there are also some cases in which our approximation brings even better accuracy. Even in the cases with accuracy loss, the accuracy degradation remains within 1%.

C. Hardware Performance

Next, we will present the hardware implementation results from the following aspects. 1) For the Cos&Sin unit of $\text{SFU}_{\text{fixed}}$, we compare $\text{UCU}_{\text{fixed}}$ and the fixed-point DesignWare IP cores. 2) To demonstrate the benefits of scaling factor fusion, we compare $\text{SFU}_{\text{fixed}}$ and $\text{UCU}_{\text{pro}} + \text{PPU}_{\text{pro}}$ with and without scaling factor fusion. 3) We evaluate the hardware performance of APU_{pro} with varying buffer depths. 4) To highlight the hardware efficiency of our proposed APU_{pro}, we compare it with $\text{APU}_{\text{float}}$. 5) RoPE_{pro}, $\text{RoPE}_{\text{fixed}}$ and $\text{RoPE}_{\text{float}}$ are compared at the PE level. 6) The superiority of our design over the GPU implementation is demonstrated.

TABLE VI

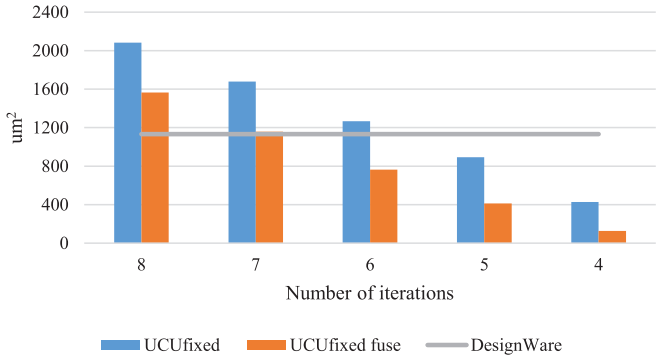
MODEL ACCURACY (%) WHEN EMPLOYING OUR PROPOSED CORDIC-BASED APPROXIMATION FOR ROPE

Task	Arc-e	Arc-c	BoolQ	Winogrande	OpenBookQA
LLaMA2-7b					
Baseline	69.32	39.93	71.04	67.17	31.60
4 iter.	69.36	39.76	70.09	67.48	31.60
4 iter. (quan.)	69.11	40.27	70.95	66.93	31.80
6 iter.	69.53	39.85	71.04	67.80	32.00
6 iter. (quan.)	69.74	40.44	70.83	67.64	31.80
8 iter.	69.44	39.59	70.95	67.25	31.40
8 iter. (quan.)	69.61	39.93	71.01	67.17	31.40
LLaMA2-13b					
Baseline	73.27	45.56	69.02	69.53	32.40
4 iter.	73.06	45.65	68.13	69.22	32.20
4 iter. (quan.)	72.77	45.48	68.41	69.46	32.20
6 iter.	73.23	46.33	69.11	70.00	32.20
6 iter. (quan.)	73.06	46.33	69.17	69.69	32.40
8 iter.	73.19	46.08	69.17	69.69	32.40
8 iter. (quan.)	72.90	45.99	69.24	69.93	32.00
Falcon-7b					
Baseline	74.66	40.19	73.52	67.09	30.60
4 iter.	74.45	39.76	74.07	67.40	30.60
4 iter. (quan.)	74.33	39.93	73.73	67.56	30.60
6 iter.	74.62	40.36	73.61	67.56	31.00
6 iter. (quan.)	74.45	40.36	73.58	67.48	31.00
8 iter.	74.62	40.10	73.64	67.48	30.60
8 iter. (quan.)	74.62	40.02	73.79	67.48	30.80

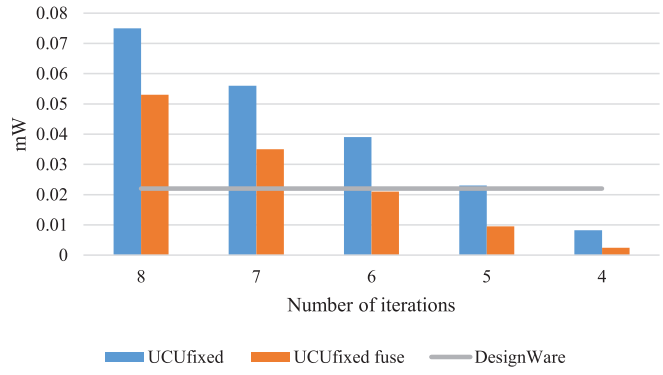
* The baseline is floating-point implementation without any approximation ($\text{RoPE}_{\text{float}}$).

In addition to $\text{UCU}_{\text{fixed}}$, the Cos&Sin unit of $\text{SFU}_{\text{fixed}}$ can also be implemented by the cosine and sine units from the DesignWare library of Synopsys. Fig. 8 provides the area and power comparisons between the two implementation approaches. Without scaling factor fusion, $\text{UCU}_{\text{fixed}}$ needs 5 iterations to outperform the DesignWare implementation in terms of both area and power. When scaling factor fusion is applied, 6 iterations are sufficient. As we have shown in Section IV-B, the performance loss brought by $5 \sim 7$ iterations is negligible. Therefore, $\text{UCU}_{\text{fixed}}$ is a more area- and power-efficient choice than the DesignWare implementation when the number of iterations is no more than 6. Henceforth, $\text{UCU}_{\text{fixed}}$ is employed in $\text{SFU}_{\text{fixed}}$, unless otherwise specified.

Fig. 8 has shown that scaling factor fusion can efficiently reduce the area cost and power consumption of $\text{UCU}_{\text{fixed}}$. Now, we present additional experimental results to further

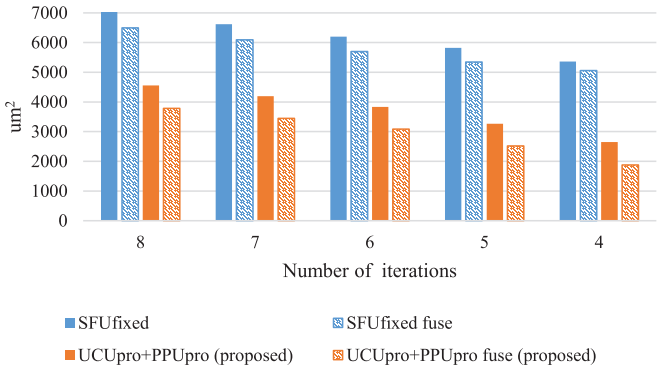


(a) Area Comparison.

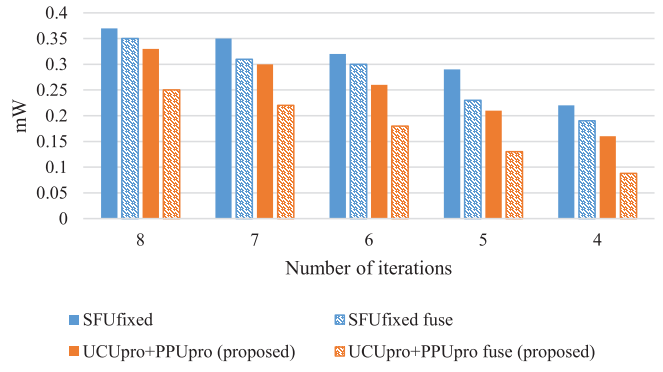


(b) Power Comparison.

Fig. 8. The area and power comparisons between $\text{UCU}_{\text{fixed}}$ and the units from the DesignWare library of Synopsys. The cases with and without scaling factor fusion are both considered for $\text{UCU}_{\text{fixed}}$. The clock frequency is set to 100MHz.



(a) Area Comparison.



(b) Power Comparison.

Fig. 9. The area and power comparisons between $\text{SFU}_{\text{fixed}}$ and $\text{UCU}_{\text{pro}} + \text{PPU}_{\text{pro}}$. The cases with and without scaling factor fusion are both considered. The clock frequency is set to 100MHz.

demonstrate the benefits of scaling factor fusion. As shown in Fig. 7, $\text{SFU}_{\text{fixed}}$ is a counterpart of $\text{UCU}_{\text{pro}} + \text{PPU}_{\text{pro}}$. For both $\text{UCU}_{\text{fixed}}$ and UCU_{pro}, the multipliers corresponding to s can be removed if the scaling factor fusion is applied. Fig. 9 shows the area and power comparisons between $\text{SFU}_{\text{fixed}}$ and $\text{UCU}_{\text{pro}}+\text{PPU}_{\text{pro}}$ with and without the scaling factor fusion. As shown in Fig. 9(a), $\text{UCU}_{\text{pro}} + \text{PPU}_{\text{pro}}$ can save 35.5%~50.6% area cost when compared with the straightforward architecture $\text{SFU}_{\text{fixed}}$. For both of them, scaling factor fusion can further reduce the area cost. When scaling factor fusion is applied, the area reduction ratio introduced by $\text{UCU}_{\text{pro}} + \text{PPU}_{\text{pro}}$ increases to 41.7%~62.9%. Fig. 9(b) provides the power comparison. Similarly, scaling factor fusion can make a further improvement for both architectures. When it is applied, $\text{UCU}_{\text{pro}} + \text{PPU}_{\text{pro}}$ can save 28.6%~53.7% power when compared with $\text{SFU}_{\text{fixed}}$.

In APU_{pro}, we define the buffer depth as d, indicating its capability to store d pairs of γ and l. In other words, d needs to be larger than the maximum possible value of D/P. Buffer depth significantly influences the hardware performance. Fig. 10 shows the area cost and power consumption of APU_{pro} with different values of d. Both of them increase with larger d. Hence, it is preferable to increase P when given a fixed D. For example, D of LLaMA2-7b, LLaMA2-13b, and Falcon-7b is 64, 64, and 32, respectively. One can set P to 32 if the three models are required to be supported simultaneously.

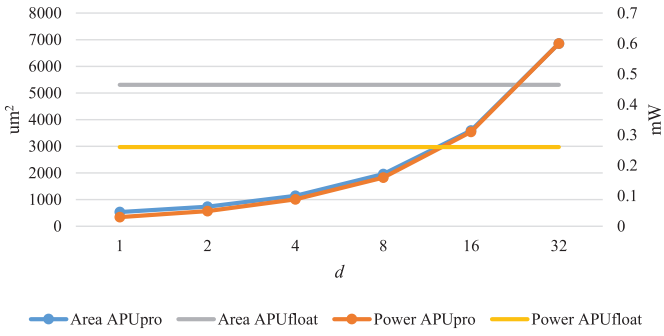


Fig. 10. Area cost and power consumption of $\text{APU}_{\text{float}}$ and APU_{pro} with different values of d. The clock frequency is set to 100MHz.

Then the maximum possible value of D/P is only 2. Therefore, setting d to 2 is sufficient.

Now we will demonstrate that our proposed solution to address the issue of excessively large input angle ranges is hardware-friendly, which means that the introduced hardware overhead is acceptable. While $\text{APU}_{\text{float}}$ is composed of only a 32-bit floating-point multiplier, APU_{pro} adopts an iterative updating rule, as given in (17). APU_{pro} is designed after exploring the mathematical relationship between input angles at adjacent positions. Such mathematical relationship is specific to RoPE. In other applications of CORDIC algorithm, the input angles are likely to be random, rather than following

TABLE VII
AREA AND POWER COMPARISONS AMONG RoPE_{PRO}, $\text{RoPE}_{\text{FIXED}}$ AND $\text{RoPE}_{\text{FLOAT}}$ AT THE PE LEVEL

Design	RoPE_{pro} (No Fusion)			RoPE_{pro} (Fusion)			$\text{RoPE}_{\text{fixed}}$ (Fusion)			RoPE_{pro} (Fusion)			$\text{RoPE}_{\text{float}}$
# of iter.	8	5	4	8	5	4	8	5	4	8	5	4	-
Performance Loss*	AZ	N	A	AZ	N	A	AZ	N	A	AZ	N	A	-
Clock Freq. (MHz)	100									20**			
Area (um^2)	5390.0	4062.4	3445.6	4581.6	3294.4	2676.0	7531.6	6070.0	5784.8	4581.6	3294.4	2676.0	38352.4
Power (mW)	0.41	0.28	0.22	0.33	0.20	0.15	0.47	0.29	0.23	0.065	0.040	0.030	0.36

* AZ: almost zero; N: negligible; A: acceptable.
** Due to a longer critical path, clock frequency of $\text{RoPE}_{\text{float}}$ cannot be set to 100 MHz. Instead, 20 MHz is a reasonable choice.
*** For APU_{pro}, we set $d = 2$.

TABLE VIII
AREA BREAKDOWN (um^2) FOR RoPE_{PRO}, $\text{RoPE}_{\text{FIXED}}$ AND $\text{RoPE}_{\text{FLOAT}}$ AT THE PE LEVEL

Design	RoPE_{pro} (Fusion, 4 iter.)	$\text{RoPE}_{\text{fixed}}$ (Fusion, 4 iter.)	$\text{RoPE}_{\text{float}}$
APU	738.0	738.0	5299.6
UCU	1535.2	-	-
Cos&Sin	-	126.4	24611.2
Others	402.8	4920.4	8441.6
Overall	2676.0	5784.8	38352.4

* Clock frequency of RoPE_{pro} and $\text{RoPE}_{\text{fixed}}$ is set to 100 MHz. Clock frequency of $\text{RoPE}_{\text{float}}$ is set to 20 MHz.

regular patterns similar to those in (17). Fig. 10 indicates that the hardware overhead of APU_{pro} is significantly lower than that of $\text{APU}_{\text{float}}$ when $d \leq 8$. As d increases, the buffers will dominate the area cost and power consumption of APU_{pro}.

Table VII compares RoPE_{pro}, $\text{RoPE}_{\text{fixed}}$ and $\text{RoPE}_{\text{float}}$ at the PE level. It indicates that our proposed RoPE_{pro} is superior in both area cost and power consumption when compared with the other two designs. After adopting the scaling factor fusion, the advantage becomes more pronounced. In the case of 5 iterations, our design can save up to 45.7% area cost and 31.0% power consumption when compared with $\text{RoPE}_{\text{fixed}}$. As shown in Table IV and Table V, they have almost the same model performance in this case. $\text{RoPE}_{\text{float}}$ suffers from significant hardware overhead. Moreover, it has a much longer critical path than RoPE_{pro} and $\text{RoPE}_{\text{fixed}}$. With 5 iterations, RoPE_{pro} can reduce up to 91.4% area cost and 88.9% power consumption compared to $\text{RoPE}_{\text{float}}$. Table VIII shows the area breakdown. It indicates that even if the Cos&Sin units are removed, the area cost of our proposed RoPE_{pro} is still lower than that of $\text{RoPE}_{\text{fixed}}$ or $\text{RoPE}_{\text{float}}$.

To showcase the acceleration effect of our design, we also compare RoPE_{pro} with a GPU. Table IX lists the speedups over the GPU for different models. Considering that LLM processes only one token at a time during the generation stage, we set $T = 1$ in the experiments. On the other hand, P and d are set to 32 and 2, respectively. For LLaMA2-7b, LLaMA2-13b and Falcon-7b, our design requires 2, 2, and 1 clock cycles, respectively. The clock frequency is set to 100 MHz, consistent with Table VII. As shown in Table IX, the speedups achieved by our design exceed two orders of magnitude.

TABLE IX
SPEEDUPS OVER GPU FOR DIFFERENT MODELS

Model	LLaMA2-7b	LLaMA2-13b	Falcon-7b
Speedup	518×	390×	609×

D. Effects at the LLM Accelerator Level

LLMs can be categorized into three classes: encoder-only, encoder-decoder and decoder-only. In recent years, decoder-only models have been gradually dominating the development of LLMs [42]. The recently released LLMs that adopt RoPE are all decoder-only models, such as PaLM, PaLM2, LLaMA, LLaMA2, and Falcon. The inference process of decoder-only LLMs consists of two stages, which can be referred to as prompting and generation, respectively. During the prompting stage, prompt sentences are fed into the model and all the tokens are processed simultaneously. However, computation in the generation stage becomes token-wise. Namely, only one token is processed at each time step. Our proposed RoPE architecture is capable of handling T tokens and P dimensions simultaneously, as depicted in Fig. 4. If $T > 1$, it may suffer from severe hardware under-utilization during the generation stage. In this case, several methods can be employed to mitigate the under-utilization issue. For example, one could process both $\mathbf{q}_i$ and $\mathbf{k}_i$ simultaneously, or handle multiple batches at once. However, the feasibility of these methods is closely tied not only to the RoPE unit but also to the other components of an LLM accelerator, the research for which is beyond the scope of this paper. Note that our design imposes no restrictions on the value of T. Even with the aforementioned methods to alleviate the under-utilization issue, our proposed RoPE architecture requires only minimal modifications, such as adding a few multiplexers, which demonstrates its high flexibility. One can set $T = 1$ in order to ensure 100% hardware utilization. Additionally, T can be increased to achieve higher parallelism in the prompting stage.

Without integrating an RoPE module into an LLM accelerator, the query and key vectors need to be transferred from

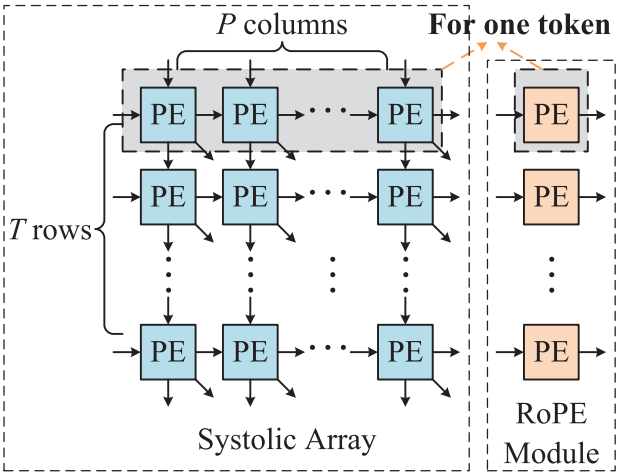


Fig. 11. Top-level architectures of the systolic array and its equipped RoPE module.

on-chip to off-chip for rotation transformation. The energy cost of off-chip operations is significantly higher than that of on-chip operations. For example, the energy cost per fetch in off-chip DRAM is typically two orders of magnitude larger than that of an on-chip multiplication [41]. In addition, Table IX has revealed that our proposed design is faster than the GPU implementation. Consequently, integrating an RoPE module rather than leaving the rotation transformation off-chip is expected to yield higher energy efficiency and speed. However, there is currently no reported research on the hardware architecture design for RoPE. Our work presents an efficient hardware architecture design for RoPE that offers lower area cost and power consumption compared to straightforward implementations, as shown in Table VII.

The RoPE module can be designed to process query and key vectors immediately after they are output by other modules of LLM accelerators. In this case, introducing extra read and write operations to on-chip memory is unnecessary. The RoPE module itself does not lead to a significant increase in latency. As shown in Table VII, our proposed RoPE architecture can operate with a clock period of 10ns. In fact, our experiments demonstrated that it can achieve a minimum clock period of less than 5ns. Moreover, a pipeline architecture in LLM accelerators can effectively mask the latency of the RoPE module.

Now, we will discuss the benefits of our design in terms of area cost at the accelerator level. Many reported works on LLM accelerators adopt systolic arrays to handle the matrix multiplications [13], [14], [17], [18], [43], [44]. The systolic array usually dominates the overall area cost of the computing modules. As illustrated in Fig. 11, a systolic array comprises $T \times P$ PEs, with each row corresponding to one token. Our proposed RoPE architecture also includes $T \times P$ PEs, with each row also corresponding to one token (see Fig. 4). Note that the P columns of PEs in our proposed RoPE architectures correspond to $2P$ dimensions, while the P columns of PEs in the systolic array correspond to P dimensions. Taking $\mathbf{q}_i$ as an example for discussion, the same applies to $\mathbf{k}_i$. When the systolic array generates the elements of $\mathbf{q}_i$, each row of PEs outputs only one element of $\mathbf{q}_i$ per cycle. Since one PE of our proposed RoPE architecture processes two adjacent elements of $\mathbf{q}_i$, employing one RoPE PE per row of PEs in the systolic array is sufficient. In other words, a $T \times P$ systolic array will be equipped with an RoPE module comprising $T \times 1$ PEs. Table X lists the ratios of the area costs between RoPE module and the systolic arrays used in [17] and [43]:

$$R = \frac{\text{Area}_{\text{RoPE}}}{\text{Area}_{\text{Systolic Array}}}. \tag{18}$$

To some extent, the value of this ratio can reflect the importance of the RoPE module within the entire accelerator, since the systolic array typically dominates the overall area cost of the computing modules.

While the systolic array of [43] utilizes 8-bit multipliers, the multipliers in the systolic array of [17] are 4-bit. 8-bit multiplications can be performed by decomposing them into 4-bit multiplications, shifting the products and then adding them together. Therefore, the systolic array of [17] is a precision-scalable design that can efficiently support both 4-bit and 8-bit matrix multiplications. In general, the area ratio increases when P decreases. Moreover, the area ratio over the 8-bit systolic array is considerably smaller than that over the 4-bit systolic array. For all the cases shown in Table X, replacing $\text{RoPE}_{\text{float}}$ with RoPE_{pro} decreases the area ratio by 9.72% to 251.19%. The area cost of $\text{RoPE}_{\text{float}}$ is even

TABLE X
THE RATIO (%) OF THE AREA COSTS BETWEEN ROPE MODULE AND THE SYSTOLIC ARRAY

Design*	8-bit array [43]**		4-bit array [17]***	
Array Size	64×64	32×32	64×64	32×32
RoPE_{pro} 8 iter.	2.61	5.22	15.93	31.87
RoPE_{pro} 5 iter.	1.26	2.51	11.54	23.08
RoPE_{pro} 4 iter.	0.95	1.91	9.34	18.68
$\text{RoPE}_{\text{fixed}}$ 8 iter.	3.62	7.23	25.82	51.64
$\text{RoPE}_{\text{fixed}}$ 5 iter.	2.46	4.92	21.43	42.86
$\text{RoPE}_{\text{fixed}}$ 4 iter.	2.31	4.62	20.33	40.66
$\text{RoPE}_{\text{float}}$	12.33	24.67	134.93	269.87

* Area, power and clock frequency of the systolic arrays of [43] and [17] have been scaled to 65-nm technology based on the scaling equations used in [43].

** When compared with the systolic array of [43], the scaled clock frequency of the systolic array is 215MHz. When the number of iterations is set to 8, the clock frequency of RoPE_{pro} and $\text{RoPE}_{\text{fixed}}$ is set to 208MHz and 196MHz, respectively. With other numbers of iterations, their clock frequencies are set to 215MHz. The clock frequency of $\text{RoPE}_{\text{float}}$ is set to 20MHz.

*** When compared with the systolic array of [17], the scaled clock frequency of the systolic array is 86MHz. Apart from $\text{RoPE}_{\text{float}}$, whose clock frequency is set to 20MHz, all other modules operate at a clock frequency of 86MHz.

approximately 2.7 times that of the 32×32 systolic array in [17]. Using our proposed RoPE_{pro} with 4 iterations, the area ratio dramatically decreases to only 18.68%. Such a decrease implies a significant saving in the overall area cost of the accelerator.

In summary, our proposed RoPE architecture can efficiently enhance the hardware performance of the entire accelerator. Furthermore, as the first reported hardware architecture design for RoPE, our work also provides important insights for future research in related fields.

V. Conclusion

In this paper, we present an efficient hardware architecture design for RoPE of LLMs, for the first time. We investigate the similarities between CORDIC algorithm and RoPE, while also taking the quantization scheme into account. A hardware-friendly solution is additionally proposed to address the issue of excessively large input angle range. Then we present a CORDIC-based approximation for RoPE. Based on the CORDIC-based approximation, an efficient hardware architecture is further developed. Experimental results demonstrate that our design is superior to the straightforward implementations in terms of area cost and power consumption.

References

[1] *OpenAI*. Accessed: Nov. 2022. [Online]. Available: https://openai.com/blog/chatgpt/

[2] W. Xin Zhao et al., "A survey of large language models," 2023, *arXiv:2303.18223*.

[3] T. B. Brown et al., "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 1877–1901.

[4] J. Achiam et al., "GPT-4 technical report," 2023, *arXiv:2303.08774*.

[5] S. Zhang et al., "OPT: Open pre-trained transformer language models," 2022, *arXiv:2205.01068*.

[6] A. Chowdhery et al., "PaLM: Scaling language modeling with pathways," *J. Mach. Learn. Res.*, vol. 24, no. 240, pp. 1–113, 2023.

[7] R. Anil et al., "PaLM 2 technical report," 2023, *arXiv:2305.10403*.

[8] H. Touvron et al., "LLaMA: Open and efficient foundation language models," 2023, *arXiv:2302.13971*.

[9] H. Touvron et al., "Llama 2: Open foundation and fine-tuned chat models," 2023, *arXiv:2307.09288*.

[10] G. Penedo et al., "The RefinedWeb dataset for falcon LLM: Outperforming curated corpora with web data only," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 79155–79172.

[11] A. Vaswani, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017.

[12] T. Lin, Y. Wang, X. Liu, and X. Qiu, "A survey of transformers," *AI Open*, vol. 3, pp. 111–132, Jan. 2022.

[13] L. Lu et al., "Sanger: A co-design framework for enabling sparse attention using reconfigurable architecture," in *Proc. 54th Annu. IEEE/ACM Int. Symp. Microarchitecture*, Oct. 2021, pp. 977–991.

[14] T. Yang et al., "DTATrans: Leveraging dynamic token-based quantization with accuracy compensation mechanism for efficient transformer architecture," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 42, no. 2, pp. 509–520, Feb. 2023.

[15] Z. Zhou, J. Liu, Z. Gu, and G. Sun, "Energon: Toward efficient acceleration of transformers using dynamic sparse attention," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 42, no. 1, pp. 136–149, Jan. 2023.

[16] H. Wang, Z. Zhang, and S. Han, "SpAtten: Efficient sparse attention architecture with cascade token and head pruning," in *Proc. IEEE Int. Symp. High-Perform. Comput. Archit. (HPCA)*, Feb. 2021, pp. 97–110.

[17] C. Guo et al., "OliVe: Accelerating large language models via hardware-friendly outlier-victim pair quantization," in *Proc. 50th Annu. Int. Symp. Comput. Archit.*, Jun. 2023, pp. 1–15.

[18] W. Li, A. Hu, N. Xu, and G. He, "Quantization and hardware architecture co-design for matrix-vector multiplications of large language models," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 71, no. 6, pp. 2858–2871, Jun. 2024.

[19] P. Shaw, J. Uszkoreit, and A. Vaswani, "Self-attention with relative position representations," 2018, *arXiv:1803.02155*.

[20] O. Press, N. A. Smith, and M. Lewis, "Train short, test long: Attention with linear biases enables input length extrapolation," 2021, *arXiv:2108.12409*.

[21] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, "RoFormer: Enhanced transformer with rotary position embedding," 2021, *arXiv:2104.09864*.

[22] J. E. Volder, "The CORDIC trigonometric computing technique," *IRE Trans. Electron. Comput.*, vol. EC-8, no. 3, pp. 330–334, Sep. 1959.

[23] J. S. Walther, "The story of unified cordic," *J. VLSI Signal Process. Syst. Signal Image Video Technol.*, vol. 25, pp. 107–112, Jun. 2000.

[24] Y. Luo, Y. Wang, Y. Ha, Z. Wang, S. Chen, and H. Pan, "Generalized hyperbolic CORDIC and its logarithmic and exponential computation with arbitrary fixed base," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 27, no. 9, pp. 2156–2169, Sep. 2019.

[25] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "SmoothQuant: Accurate and efficient post-training quantization for large language models," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 38087–38099.

[26] W. Shao et al., "OmniQuant: Omnidirectionally calibrated quantization for large language models," 2023, *arXiv:2308.13137*.

[27] W. Wang et al., "Model compression and efficient inference for large language models: A survey," 2024, *arXiv:2402.09748*.

[28] Z. Yuan et al., "LLM inference unveiled: Survey and roofline model insights," 2024, *arXiv:2402.16363*.

[29] N. Shazeer, "Fast transformer decoding: One write-head is all you need," 2019, *arXiv:1911.02150*.

[30] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, "GQA: Training generalized multi-query transformer models from multi-head checkpoints," 2023, *arXiv:2305.13245*.

[31] S. Merity, C. Xiong, J. Bradbury, and R. Socher, "Pointer sentinel mixture models," 2016, *arXiv:1609.07843*.

[32] C. Raffel et al., "Exploring the limits of transfer learning with a unified text-to-text transformer," *J. Mach. Learn. Res.*, vol. 21, no. 140, pp. 1–67, 2020.

[33] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate post-training quantization for generative pre-trained transformers," 2022, *arXiv:2210.17323*.

[34] Z. Yuan et al., "RPTQ: Reorder-based post-training quantization for large language models," 2023, *arXiv:2304.01089*.

[35] C. Lee, J. Jin, T. Kim, H. Kim, and E. Park, "OWQ: Outlier-aware weight quantization for efficient fine-tuning and inference of large language models," 2023, *arXiv:2306.02272*.

[36] T. Dettmers et al., "SpQR: A sparse-quantized representation for near-lossless LLM weight compression," 2023, *arXiv:2306.03078*.

[37] P. Clark et al., "Think you have solved question answering? Try ARC, the AI2 reasoning challenge," 2018, *arXiv:1803.05457*.

[38] C. Clark, K. Lee, M.-W. Chang, T. Kwiatkowski, M. Collins, and K. Toutanova, "BoolQ: Exploring the surprising difficulty of natural yes/no questions," 2019, *arXiv:1905.10044*.

[39] K. Sakaguchi, R. L. Bras, C. Bhagavatula, and Y. Choi, "WinoGrande: An adversarial Winograd schema challenge at scale," *Commun. ACM*, vol. 64, no. 9, pp. 99–106, 2021.

[40] T. Mihaylov, P. Clark, T. Khot, and A. Sabharwal, "Can a suit of armor conduct electricity? A new dataset for open book question answering," 2018, *arXiv:1809.02789*.

[41] S. Han et al., "EIE: Efficient inference engine on compressed deep neural network," in *Proc. 43rd IEEE Int. Symp. Comput. Archit.*, May 2016, pp. 243–254.

[42] J. Yang et al., "Harnessing the power of LLMs in practice: A survey on ChatGPT and beyond," *ACM Trans. Knowl. Discovery Data*, vol. 18, no. 6, pp. 1–32, Jul. 2024.

[43] W. Li, A. Hu, N. Xu, and G. He, "CoDA: A co-design framework for versatile and efficient attention accelerators," *IEEE Trans. Comput.*, vol. 73, no. 8, pp. 1924–1938, Aug. 2024.

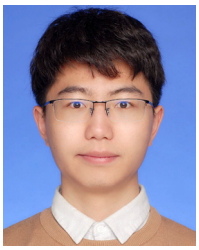
[44] F. Liu et al., "SPARK: Scalable and precision-aware acceleration of neural networks via efficient encoding," in *Proc. IEEE Int. Symp. High-Performance Comput. Archit. (HPCA)*, Mar. 2024, pp. 1029–1042.

[45] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," 2019, *arXiv: 1912.01703*.

[46] T. Wolf et al., "HuggingFace's transformers: State-of-the-art natural language processing," 2019, *arXiv:1910.03771*.

Wenjie Li received the B.E. and M.S. degrees from Nanjing University, Nanjing, China, in 2017 and 2020, respectively, and the Ph.D. degree from Shanghai Jiao Tong University, Shanghai, China, in 2025.

He has published more than ten peer-reviewed articles. His previous research interests include error correction codes and stochastic computing. His current research interests include VLSI design for deep learning.



Gang Wang received the B.S. degree in microelectronics science and engineering and the M.S. degree in integrated circuit engineering from Shanghai Jiao Tong University, Shanghai, China, in 2019 and 2022, respectively, where he is currently pursuing the Ph.D. degree.

His current research interests include domain-specific architecture for autonomous driving and artificial intelligence (AI) accelerators.

Dongxu Lyu received the B.S. degree in microelectronics science and engineering from Shanghai Jiao Tong University, Shanghai, China, in 2020, where he is currently pursuing the Ph.D. degree with the School of Electronic Information and Electrical Engineering.

His current research interests include the intersection of autonomous driving systems, and digital circuits and systems. The current focus is application-specific integrated circuit (ASIC) design for artificial intelligence (AI) applications, such as 3-D perception for autonomous driving.

Ningyi Xu received the B.S. and Ph.D. degrees from Tsinghua University, Beijing, China, in 2001 and 2006, respectively.

He was a Principle Architect with Baidu, and a Lead Researcher with the Hardware Computing Group, Microsoft Research Asia, Beijing. He is currently a Professor with the Qingyuan Research Institute, Shanghai Jiao Tong University, Shanghai, China. His current research interests include domain-specific computing, computer architecture, parallel computing, and machine learning systems.

Guanghui He (Member, IEEE) received the B.S. degree in electronic engineering from the University of Electronic Science and Technology of China, Chengdu, China, in 2002, and the Ph.D. degree in electronic engineering from Tsinghua University, Beijing, China, in 2007.

He is currently a Professor with the Department of Micro/Nano Electronics and the MoE Key Laboratory of Artificial Intelligence, Shanghai Jiao Tong University. His research interests include energy efficient algorithms and circuits design for wireless communication, image processing, emerging memory devices, and artificial intelligent systems.